

7.3.1 Jacobi iteration ¶

7.3.1 Jacobi iteration Part 1

fit width



Let's review what we saw in Subsection 7.1.1. The linear system $Au = f$

$$\begin{array}{cccccccccccl} 4v_0 & -v_1 & & & -v_4 & & & & & & = h^2\phi_0 \\ -v_0 & +4v_1 & -v_2 & & & -v_5 & & & & & = h^2\phi_1 \\ & -v_1 & +4v_2 & -v_3 & & & -v_6 & & & & = h^2\phi_2 \\ & & -v_2 & +4v_3 & & & & -v_7 & & & = h^2\phi_3 \\ -v_0 & & & & +4v_4 & -v_5 & & & -v_8 & & = h^2\phi_4 \\ & \ddots & & & & \ddots & \ddots & \ddots & & \ddots & \vdots \end{array}$$

which can be written in matrix form as

$$\left(\begin{array}{ccc|ccc|ccc} 4 & -1 & & -1 & & & & & & \\ -1 & 4 & -1 & & -1 & & & & & \\ & -1 & 4 & -1 & & & & & & \\ & & -1 & 4 & & & -1 & & & \\ \hline -1 & & & & 4 & -1 & & -1 & & \\ & -1 & & & -1 & 4 & -1 & & \ddots & \\ & & -1 & & & -1 & 4 & -1 & & \\ \hline & & & -1 & & & & 4 & \ddots & \\ & & & & \ddots & & & \ddots & \ddots & \end{array}\right) \begin{pmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \\ v_7 \\ v_8 \\ \vdots \end{pmatrix} = \begin{pmatrix} h^2\phi_0 \\ h^2\phi_1 \\ h^2\phi_2 \\ h^2\phi_3 \\ h^2\phi_4 \\ h^2\phi_5 \\ h^2\phi_6 \\ h^2\phi_7 \\ h^2\phi_8 \\ \vdots \end{pmatrix}.$$

was solved by repeatedly updating

$$v_i = \frac{h^2\phi_i + v_{i-N} + v_{i-1} + v_{i+1} + v_{i+N}}{4}$$

modified appropriately for points adjacent to the boundary. Let's label the value of v_i during the k th iteration with $v_i^{(k)}$ and state the algorithm more explicitly as

```
for  $k = 0, \dots$ , convergence
  for  $i = 0, \dots, N \times N - 1$ 
     $v_i^{(k+1)} = (h^2\phi_i + v_{i-N}^{(k)} + v_{i-1}^{(k)} + v_{i+1}^{(k)} + v_{i+N}^{(k)})/4$ 
  endfor
endfor
```

again, modified appropriately for points adjacent to the boundary. The superscripts are there to emphasize the iteration during which a value is updated. In practice, only the values for iteration k and $k + 1$ need to be stored. We can also capture the algorithm with a vector and matrix as

$$\begin{array}{cccccccccl} 4v_0^{(k+1)} & = & v_1^{(k)} & & & +v_4^{(k)} & & & & +h^2\phi_0 \\ 4v_1^{(k+1)} & = & v_0^{(k)} & & +v_2^{(k)} & & & +v_5^{(k)} & & +h^2\phi_1 \\ 4v_2^{(k+1)} & = & & v_1^{(k)} & & +v_3^{(k)} & & & +v_6^{(k)} & +h^2\phi_2 \\ 4v_3^{(k+1)} & = & & & v_2^{(k)} & & & & +v_7^{(k)} & +h^2\phi_3 \\ 4v_4^{(k+1)} & = & v_0^{(k)} & & & & +v_5^{(k)} & & -v_8^{(k)} & +h^2\phi_4 \\ \vdots & & \ddots & & & \ddots & \ddots & \ddots & \ddots & \vdots \end{array}$$

which can be written in matrix form as

$$\begin{pmatrix} 4 & & & \\ & 4 & & \\ & & 4 & \\ & & & 4 \\ \hline & & 4 & \\ & & & 4 \\ & & & & 4 \\ \hline & & & & & 4 \\ & & & & & & \ddots \end{pmatrix} \begin{pmatrix} v_0^{(k+1)} \\ v_1^{(k+1)} \\ v_2^{(k+1)} \\ v_3^{(k+1)} \\ \hline v_4^{(k+1)} \\ v_5^{(k+1)} \\ v_6^{(k+1)} \\ v_7^{(k+1)} \\ \hline v_8^{(k+1)} \\ \vdots \end{pmatrix} \quad (7.3.1)$$

$$= \begin{pmatrix} 0 & 1 & & & & & & \\ 1 & 0 & 1 & & & & & \\ & 1 & 0 & 1 & & & & \\ & & 1 & 0 & & & & \\ \hline 1 & & & & 0 & 1 & & 1 \\ & 1 & & & 1 & 0 & 1 & \ddots \\ & & 1 & & 1 & 0 & 1 & \\ \hline & & & 1 & & & 0 & \ddots \\ & & & & \ddots & & \ddots & \ddots \end{pmatrix} \begin{pmatrix} v_0^{(k)} \\ v_1^{(k)} \\ v_2^{(k)} \\ v_3^{(k)} \\ \hline v_4^{(k)} \\ v_5^{(k)} \\ v_6^{(k)} \\ v_7^{(k)} \\ \hline v_8^{(k)} \\ \vdots \end{pmatrix} + \begin{pmatrix} h^2 \phi_0 \\ h^2 \phi_1 \\ h^2 \phi_2 \\ h^2 \phi_3 \\ \hline h^2 \phi_4 \\ h^2 \phi_5 \\ h^2 \phi_6 \\ h^2 \phi_7 \\ \hline h^2 \phi_8 \\ \vdots \end{pmatrix}.$$

7.3.1 Jacobi iteration, Part 2

fit width



How can we capture this more generally?

- We wish to solve $Ax = y$.

We write A as the difference of its diagonal, $M = D$, and the negative of its off-diagonal part, $N = D - A$ so that

$$A = D - (D - A) = M - N.$$

In our example, $M = 4I$ and $N = 4I - A$.

- We then notice that

$$Ax = y$$

can be rewritten as

$$(M - N)x = y$$

or, equivalently,

$$Mx = Nx + y.$$

If you think about it carefully, this captures (7.3.1) for our example. Finally,

$$x = M^{-1}(Nx + y).$$

- If we now let $x^{(k)}$ be the values of our vector x in the current step. Then the values after all elements have been updated are given by the vector

$$x^{(k+1)} = M^{-1}(Nx^{(k)} + y).$$

- All we now need is an initial guess for the solution, $x^{(0)}$, and we are ready to iteratively solve the linear system by computing $x^{(1)}$, $x^{(2)}$, etc., until we (approximately) reach a fixed point where $x^{(k+1)} = M^{-1}(Nx^{(k)} + y) \approx x^{(k)}$.

The described method, where M equals the diagonal of A and $N = D - A$, is known as the **Jacobi iteration**.

Remark 7.3.1.1. The important observation is that the computation involves a matrix-vector multiplication with a sparse matrix, $N = D - A$, and a solve with a diagonal matrix, $M = D$.